Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №7

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 7
Листинг: 8
main7.c 8
script.sh 9
Makefile 9

Задание: разработайте программу с динамической загрузкой библиотеки совместного доступа, созданную в лабораторной 5. Проконтролируйте загрузку и выгрузку библиотеки, используя файл:

/proc/<идентификатор процесса>/maps

Цель: знакомство со структурой адресного пространства процесса

Ход работы:

Сначала разработаем программу, которая будет загружать библиотеку и использовать её (код программы представлен в листинге main7.c). Для компиляции программы напишем makefile (код также представлен в листинге).

```
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ make
gcc -Wall -g -o dynamic_student_program main7.c -ldl
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ ./dynamic_student_program
Студент добавлен в журнал.
Студент добавлен в журнал.
Студент добавлен в журнал.

Журнал успеваемости студентов:
Имя: Пётр, Номер: 103, Оценка: 78.20
Имя: Мария, Номер: 102, Оценка: 92.00
Имя: Иван, Номер: 101, Оценка: 85.50
Студент удален из журнала.

Журнал успеваемости студентов:
Имя: Пётр, Номер: 103, Оценка: 78.20
Имя: Иван, Номер: 101, Оценка: 85.50
Сериализация завершена.
```

Рис.1 – Компиляция, запуск и результат работы программы.

Для контролирования загрузки/выгрузки библиотеки был написан простейший bash скрипт (код представлен в листинге). Были добавлены точки остановки в программу и вывод PID (47-49 и 54 строки):

```
47      printf("\nPID %d\n", getpid());
48      int flag_1, flag_2;
49      scanf("%d", &flag_1);
50
51      // Закрытие библиотеки
52      dlclose(lib_handle);
53
54      scanf("%d", &flag_2);
55
56      return 0;
57  }
58
```

Рис. 2 – Точки остановки для проверки загрузки/выгрузки библиотеки и вывод PID.

Для проверки загрузки/выгрузки нужно произвести следующие действия:

1. Скомпилировать и запустить программу.
2. Скопировать выведенный PID и вставить его в bash скрипт.

3. Запустить bash скрипт. (Выведется информация о загрузке библиотеки).
4. Ввести любое число для продолжения программы (закрытия библиотеки).
5. Запустить bash скрипт. (Вывод bash скрипта будет пустым т.к. библиотека была выгружена).
6. Ввести любое число для продолжения (завершения) программы.

```
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ make
gcc -Wall -g -o app main7.c -ldl
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ ./app
Студент добавлен в журнал.
Студент добавлен в журнал.
Студент добавлен в журнал.

Журнал успеваемости студентов:
Имя: Пётр, Номер: 103, Оценка: 78.20
Имя: Мария, Номер: 102, Оценка: 92.00
Имя: Иван, Номер: 101, Оценка: 85.50
Студент удален из журнала.

Журнал успеваемости студентов:
Имя: Пётр, Номер: 103, Оценка: 78.20
Имя: Иван, Номер: 101, Оценка: 85.50
Сериализация завершена.

PID 10085
1
1
nothomepc@DESKTOP-M12GO9R:~/code/lab7$
```

Рис. 3 – Компиляция, запуск и результат работы программы с точками остановки.

```
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ sh script.sh
7fce46f7d000-7fce46f7e000 r--p 00000000 08:20 18410                      /home/nothomepc/code/lab7/libstudents.so
7fce46f7e000-7fce46f7f000 r-xp 00001000 08:20 18410                      /home/nothomepc/code/lab7/libstudents.so
7fce46f7f000-7fce46f80000 r--p 00002000 08:20 18410                      /home/nothomepc/code/lab7/libstudents.so
7fce46f80000-7fce46f81000 r--p 00002000 08:20 18410                      /home/nothomepc/code/lab7/libstudents.so
7fce46f81000-7fce46f82000 rw-p 00003000 08:20 18410                      /home/nothomepc/code/lab7/libstudents.so
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ sh script.sh
nothomepc@DESKTOP-M12GO9R:~/code/lab7$ sh script.sh
grep: /proc/10085/maps: No such file or directory
nothomepc@DESKTOP-M12GO9R:~/code/lab7$
```

Рис. 4 – Запуск bash скрипта и вывод информации в трех разных случаях (1 – после загрузки библиотеки; 2 – после выгрузки библиотеки; 3 - после завершения программы).

Анализ выведенной информации из bash скрипта:

В этом выводе содержится информация о разных секциях библиотеки:

- **7f6cb8e0b000-7f6cb8e0c000**: Это диапазон виртуальных адресов в памяти процесса, занимаемых этой секцией библиотеки.
- **r--p**, **r-xp**, **r--p**, **r--p**, **rw-p**: Эти буквы обозначают права доступа к соответствующей секции. Ваш вывод показывает различные типы разрешений, такие как чтение (**r**), запись (**w**), и выполнение (**x**).
- **00000000**, **00001000**, **00002000**, **00003000**: Эти значения представляют смещения в байтах относительно начала файла **libstudents.so**. Например, **00001000** означает, что смещение этой секции в файле начинается с 4096-го байта файла.
- **/home/nothomepc/code/lab7/libstudents.so**: Это путь к файлу **libstudents.so**, который был загружен в память процесса. Этот путь указывает на местоположение вашей библиотеки на файловой системе.

Вывод:

- Мы познакомились со структурой адресного пространства процесса.
- Разработали программу с динамической загрузкой библиотеки совместного доступа, созданную в лабораторной 5.
- Проконтролировали загрузку и выгрузку библиотеки, используя файл /proc/<идентификатор процесса>/maps.

Листинг:

main7.c

```
#include <stdio.h>
#include <dlfcn.h>
#include "student.h"
#include <unistd.h>

int main() {
    void *lib_handle;
    struct Student* (*createStudent)(char[], int, float);
    void (*insertStudent)(struct Student*);
    void (*displayStudents)();
    void (*deleteStudent)(int);
    void (*serializeStudents)(const char *);

    // Загрузка библиотеки
    lib_handle = dlopen("./libstudents.so", RTLD_LAZY);
    if (!lib_handle) {
        fprintf(stderr, "Ошибка при загрузке библиотеки: %s\n", dlerror());
        return 1;
    }

    // Получение указателей на функции из библиотеки
    createStudent = dlsym(lib_handle, "createStudent");
    insertStudent = dlsym(lib_handle, "insertStudent");
    displayStudents = dlsym(lib_handle, "displayStudents");
    deleteStudent = dlsym(lib_handle, "deleteStudent");
    serializeStudents = dlsym(lib_handle, "serializeStudents");

    if (createStudent && insertStudent && displayStudents && deleteStudent &&
serializeStudents) {
        struct Student* student1 = createStudent("Иван", 101, 85.5);
        struct Student* student2 = createStudent("Мария", 102, 92.0);
        struct Student* student3 = createStudent("Пётр", 103, 78.2);

        insertStudent(student1);
        insertStudent(student2);
        insertStudent(student3);

        displayStudents();
        deleteStudent(102);
        displayStudents();

        serializeStudents("journal.dat");
    } else {
        fprintf(stderr, "Ошибка при получении указателей на функции: %s\n",
dlerror());
        return 1;
    }
```

```
    printf("\nPID %d\n", getpid());
    int flag_1, flag_2;
    scanf("%d", &flag_1);

    // Закрытие библиотеки
    dlclose(lib_handle);

    scanf("%d", &flag_2);

    return 0;
}
```

script.sh

```
PID=10085
LIBRARY_NAME=libstudents.so

grep "$LIBRARY_NAME" /proc/$PID/maps
```

Makefile

```
CC = gcc
CFLAGS = -Wall -g
LIBS = -ldl

TARGET = app

all: $(TARGET)

$(TARGET): main7.c
    $(CC) $(CFLAGS) -o $@ $^ $(LIBS)

clean:
    rm -f $(TARGET)
```